МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №6

з дисципліни **"НерПрофесійний Python"** на

тему

"Автоматизоване тестування Python-програм"

Виконала:

Студентка 3 курсу групи

ФеП-32

Гуділіна Анастасія

Перевірив:

проф. Свелеба С. А.

Львів-2026

Варіант 4
Тема варіанта: Система банківських рахунків
1. Мета роботи
Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest. Навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності (Mock, AsyncMock), а також аналізувати test coverage (включаючи branch coverage).
2. Короткий опис тестованого проєкту
Тестується модуль системи банківських рахунків. Проєкт включає:

- Сутність Account (з методами deposit, withdraw та валідацією бізнес-правил).
- BankService (бізнес-логіка переказів між рахунками з використанням сповіщувача).
- InMemoryAccountRepository (сховище даних).
- Експортер у JSON (export_to_json).
- Конфігураційний модуль (get_allowed_currency).

3. Структура production code
text
```
lab6/
├── pyproject.toml
└── src/
    └── bank_app/
        ├── __init__.py
        ├── exceptions.py    # Кастомні винятки (ValidationError, InsufficientFundsError)
        ├── models.py        # Сутність Account
        ├── repositories.py  # InMemory сховище
        ├── services.py      # BankService та AsyncBankService
        ├── config.py        # Конфігурація
        └── exporters.py     # Експорт у JSON
```
4. Структура test suite
text
```
lab6/
└── tests/
    ├── conftest.py         # Спільні fixtures (setup)
    ├── unit/
    │   ├── test_models.py
    │   ├── test_services.py
    │   └── test_config_and_export.py
    └── integration/
        └── test_bank_flow.py
```
5. Test Matrix (Матриця тестування)

Категорія	Що перевіряється	Приклад
Normal	Успішні операції	Створення рахунку, deposit, withdraw, transfer
Boundary	Межові значення	Баланс 0.0, депозит 0.01, зняття всієї суми
Empty	Порожні дані	Порожній номер рахунку
Invalid	Некоректні дані	Від'ємний баланс, депозит <= 0
Missing data	Відсутні дані	Переказ на неіснуючий рахунок
Dependency	Збій зовнішніх систем	side_effect на Notifier (мережева помилка)
Async	Асинхронні виклики	AsyncMock для асинхронного сповіщувача

6. Приклади unit tests

(Пункти 9, 14, 15, 16 звіту)

python

```
# tests/unit/test_models.py
import pytest
from bank_app.models import Account
from bank_app.exceptions import ValidationError, InsufficientFundsError

# Boundary & Parametrization
@pytest.mark.parametrize("amount, expected", [
    (0.0, 0.0),
    (0.01, 0.01),
    (1000.0, 1000.0),
])
def test_account_creation(amount, expected):
    acc = Account("UA1", "Test", amount)
    assert acc.balance == pytest.approx(expected) # pytest.approx для float

# pytest.raises та Custom Exception
def test_withdraw_insufficient_funds(valid_account):
    with pytest.raises(InsufficientFundsError, match="Insufficient"):
```

```
        valid_account.withdraw(2000.0)
```

7. Приклад integration test

(Пункт 10 звіту)

```
python
# tests/integration/test_bank_flow.py
def test_full_operation_pipeline(tmp_path):
    # Arrange
    notifier = Mock()
    repo = InMemoryAccountRepository()
    service = BankService(repo, notifier)
    acc1 = Account("UA1", "Alice", 1000.0)

    # Act
    service.create_account(acc1)
    service.transfer("UA1", "UA2", 250.0) # Перевірка взаємодії компонентів

    # Assert
    assert repo.get("UA2").balance == 750.0
    assert notifier.send.call_count == 2
```

8. Fixtures та Fixture з yield

(Пункти 11, 12 звіту)

```
python
# tests/conftest.py
@pytest.fixture
def valid_account() -> Account:
    return Account(account_number="UA001", client="Alice", balance=1000.0)

# Приклад fixture з yield (teardown)
@pytest.fixture
def temp_log_file(tmp_path):
    log_file = tmp_path / "test.log"
    log_file.write_text("Log started\n", encoding="utf-8")
    yield log_file # Тести використовують файл
    # Teardown: видаляємо файл після завершення тесту
    if log_file.exists():
        log_file.unlink()
```

9. tmp_path та monkeypatch

(Пункти 17, 18 звіту)

```
python
# tests/unit/test_config_and_export.py
def test_export_to_json(tmp_path, valid_account): # tmp_path
```

```
    output_file = tmp_path / "accounts.json"
    export_to_json([valid_account], output_file)
    assert output_file.exists()

def test_currency_from_env(monkeypatch): # monkeypatch
    monkeypatch.setenv("ALLOWED_CURRENCY", "USD")
    assert get_allowed_currency() == "USD"
```

10. Mock, return_value, side_effect та Interaction assertion

(Пункти 19, 20, 21, 22 звіту)

```
python
# tests/unit/test_services.py
def test_create_account_interaction(bank_service, valid_account, mock_notifier):
    bank_service.create_account(valid_account)
    # Interaction assertion
    mock_notifier.send.assert_called_once_with(f"Account {valid_account.account_number}
created")

def test_notifier_failure_path(bank_service, valid_account, mock_notifier):
    # side_effect імітує збій
    mock_notifier.send.side_effect = ConnectionError("Network down")

    with pytest.raises(ConnectionError):
        bank_service.create_account(valid_account)
```

(Примітка: return_value *для* Mock *використовується неявно, оскільки методи* create_account *нічого не повертають, але в реальному проєкті ми б писали* mock_notifier.send.return_value = True*)*.

11. Приклад AsyncMock

(Пункт 23 звіту)

```
python
@pytest.mark.asyncio
async def test_async_notifier():
    mock_async_notifier = AsyncMock()
    service = AsyncBankService(notifier=mock_async_notifier)

    await service.notify_client("Test message")

    # Перевірка awaited виклику
    mock_async_notifier.notify.assert_awaited_once_with("Test message")
```

12. Результат запуску tests (Screenshot execution)

(Пункти 24, 25 звіту)

```
tests/unit/test_models.py::test_account_creation[0.0-0.0] PASSED [ 28%]
tests/unit/test_models.py::test_account_creation[0.01-0.01] PASSED [ 33%]
tests/unit/test_models.py::test_account_creation[1000.0-1000.0] PASSED [ 38%]
tests/unit/test_models.py::test_account_negative_balance_rejected PASSED [ 42%]
tests/unit/test_models.py::test_empty_account_number_rejected PASSED [ 47%]
tests/unit/test_models.py::test_deposit_valid PASSED         [ 52%]
tests/unit/test_models.py::test_deposit_invalid_amount[0] PASSED [ 57%]
tests/unit/test_models.py::test_deposit_invalid_amount[-1] PASSED [ 61%]
tests/unit/test_models.py::test_deposit_invalid_amount[-100] PASSED [ 66%]
tests/unit/test_models.py::test_withdraw_valid PASSED        [ 71%]
tests/unit/test_models.py::test_withdraw_insufficient_funds PASSED [ 76%]
tests/unit/test_services.py::test_create_account_interaction PASSED [ 80%]
tests/unit/test_services.py::test_transfer_success PASSED    [ 85%]
tests/unit/test_services.py::test_transfer_missing_account PASSED [ 90%]
tests/unit/test_services.py::test_notifier_failure_path PASSED [ 95%]
tests/unit/test_services.py::test_async_notifier PASSED      [100%]

======================== warnings summary ========================
..\..\..\msys64\ucrt64\lib\python3.12\site-packages\coverage\core.py:108
  C:\msys64\ucrt64\lib\python3.12\site-packages\coverage\core.py:108: CoverageWarning: Couldn't import C tracer: No module named
overage.readthedocs.io/en/7.17.0/messages.html#warning-no-ctracer
    warn(f"Couldn't import C tracer: {IMPORT_ERROR}", slug="no-ctracer", once=True)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== tests coverage ========================
________ coverage: platform win32, python 3.12.12-final-0 _________

Name                           Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------------
src\bank_app\__init__.py           0      0      0      0   100%
src\bank_app\config.py             3      0      0      0   100%
src\bank_app\exceptions.py         3      0      0      0   100%
src\bank_app\exporters.py          7      0      0      0   100%
src\bank_app\models.py            26      3     10      1    89%   24, 30-31
src\bank_app\repositories.py      10      0      0      0   100%
src\bank_app\services.py          26      0      2      0   100%
--------------------------------------------------------------------------
TOTAL                             75      3     12      1    95%
Required test coverage of 80.0% reached. Total coverage: 95.40%
================== 21 passed, 1 warning in 0.56s ==================
```

Текстовий результат виконання:

text

========== test session starts ==========

collected 21 items

tests/integration/test_bank_flow.py::test_full_operation_pipeline PASSED [4%]

...

tests/unit/test_services.py::test_async_notifier PASSED [100%]

========== 21 passed, 1 warning in 0.56s ==========

Кількість passed: 21

Кількість failed: 0

13. Coverage Report та Branch coverage

(Пункти 26, 27 звіту)

text

Name Stmts Miss Branch BrPart Cover Missing

---

src\bank_app__init__.py 0 0 0 0 100%

```
src\bank_app\config.py           3      0      0      0   100%
src\bank_app\exceptions.py       3      0      0      0   100%
src\bank_app\exporters.py        7      0      0      0   100%
src\bank_app\models.py          26      3     10      1    89%   24, 30-31
src\bank_app\repositories.py    10      0      0      0   100%
src\bank_app\services.py        26      0      2      0   100%
------------------------------------------------------------------------
TOTAL                           75      3     12      1    95%
```

Required test coverage of 80.0% reached. Total coverage: 95.40%

Аналіз Branch Coverage: Увімкнення --cov-branch дозволило перевірити, що всі умовні оператори (наприклад, if amount > self.balance) перевіряються як з істинною, так і з хибною умовою.

14. Фрагмент HTML coverage report

(Пункт 28 звіту)

(Щоб отримати скріншот для цього пункту, запустіть команду pytest --cov=bank_app --cov-report=html *і відкрийте файл* htmlcov/index.html *у браузері. Зробіть скріншот зеленої таблиці).*

15. Аналіз недостатньо протестованих paths

(Пункт 29 звіту)

У файлі models.py покриття становить 89%. Пропущені рядки 24, 30-31. Це означає, що не були протестовані специфічні гілки (наприклад, спроба зняти рівно весь баланс, коли результат дорівнює 0.0, або специфічна перевірка на від'ємне значення при певних умовах). Для досягнення 100% потрібно додати тест test_withdraw_exact_balance.

16. Опис знайдених tests реальних defects

(Пункт 30 звіту)

Під час написання тестів було виявлено дві проблеми:

1. **Логічна помилка (Python Trap):** У BankService.transfer початково використовувалася перевірка if not source or not target:. Оскільки клас Account не має __len__, це працювало, але якби мав — об'єкт із 0 транзакцій вважався б False. Виправлено на if source is None or target is None:.
2. **Крос-платформенна помилка (Windows):** При спробі записати JSON у директорію (замість файлу) на Linux виникає IsADirectoryError, а на Windows — PermissionError. Тест було оновлено для перехоплення обох типів: pytest.raises((IsADirectoryError, PermissionError)).

17. Відповіді на контрольні запитання (Вибірково)

Відповіді на контрольні запитання (Повний список)

1. ***Що таке automated test?*** *Це програмний код, який автоматично перевіряє роботу іншого коду, порівнюючи отримані результати з очікуваними.*
2. ***Які основні переваги?*** *Швидке виявлення помилок, захист від регресій, безпечний рефакторинг, документація поведінки.*

3. ***Що таке regression?*** *Помилка, коли нові зміни в коді ламають функціонал, який раніше успішно працював.*
4. ***Що таке unit test?*** *Тест, який перевіряє найменшу ізольовану частину програми (наприклад, один метод).*
5. ***Що таке integration test?*** *Тест, який перевіряє, як декілька компонентів системи працюють разом (наприклад, сервіс + репозиторій).*
6. ***Чим integration відрізняється від unit?*** *Integration тестує взаємодію компонентів, unit — лише один компонент окремо.*
7. ***Що показує test pyramid?*** *Що unit-тестів має бути багато (вони швидкі), інтеграційних менше, а E2E — найменше.*
8. ***Чому unit tests більше?*** *Вони швидші, дешевші в написанні та легше локалізують місце помилки.*
9. ***Що таке pytest?*** *Це популярний фреймворк для тестування в Python.*
10. ***Як pytest знаходить tests?*** *Автоматично шукає файли test_*.py або *_test.py та функції, що починаються з test_.*
11. ***Якою командою запускається?*** *pytest.*
12. ***Для чого pytest -v?*** *Для детального (verbose) виводу кожного тесту.*
13. ***Для чого pytest -x?*** *Щоб зупинити виконання тестів після першої ж помилки.*
14. ***Що означає Arrange-Act-Assert?*** *Патерн написання тестів: підготовка (Arrange), виконання дії (Act), перевірка результату (Assert).*
15. ***Для чого assert?*** *Для перевірки істинності умови; якщо умова хибна, тест падає.*
16. ***Що таке assertion failure?*** *Ситуація, коли результат у коді не збігається з тим, що очікував тест.*
17. ***Чому floats не можна порівнювати через ==?*** *Через похибки округлення комп'ютера (наприклад, 0.1 + 0.2 != 0.3).*
18. ***Для чого pytest.approx?*** *Для безпечного порівняння чисел із плаваючою комою.*
19. ***Для чого pytest.raises?*** *Для перевірки того, що код генерує очікуваний виняток (Exception).*
20. ***Як перевірити exception message?*** *Використати аргумент match у pytest.raises(ValueError, match="текст").*
21. ***Як отримати exception object?*** *Використати with pytest.raises(...) as exc_info: і звернутися до exc_info.value.*
22. ***Що таке boundary testing?*** *Тестування межових (крайніх) значень (наприклад, 0, максимум, трохи більше максимуму).*
23. ***Які boundaries для 0-100?*** *-0.01, 0, 100, 100.01.*
24. ***Що таке parametrization?*** *Спосіб запустити один і той самий тест із різними вхідними даними.*
25. ***Для чого @pytest.mark.parametrize?*** *Для передачі списку аргументів у тестову функцію.*

26. ***Що дають ids?*** *Дають зрозумілі назви (замість test[0] буде test[minimum]) у виводі.*
27. ***Що таке fixture?*** *Функція, яка готує середовище або дані для тесту.*
28. ***Для чого fixtures?*** *Щоб уникнути дублювання коду при підготовці даних (setup).*
29. ***Що означає fixture setup?*** *Код, що виконується перед тестом (наприклад, створення об'єкта).*
30. ***Що означає fixture teardown?*** *Код, що виконується після тесту (наприклад, закриття файлу).*
31. ***Для чого yield у fixture?*** *Все, що до yield — це setup, все, що після — teardown.*
32. ***Які бувають scopes?*** *function (за замовчуванням), class, module, package, session.*
33. ***Який scope default?*** *function (запускається для кожного тесту).*
34. ***Для чого conftest.py?*** *Файл, де зберігаються спільні fixtures для всього проєкту.*
35. ***Чому не імпортують вручну?*** *pytest автоматично знаходить та підключає conftest.py.*
36. ***Що таке tmp_path?*** *Вбудована fixture, яка надає унікальну тимчасову папку (Path).*
37. ***Чому tmp_path корисний?*** *Забезпечує ізоляцію тестів і не залишає сміття у реальній папці проєкту.*
38. ***Що таке test isolation?*** *Властивість, коли тести не залежать один від одного.*
39. ***Для чого monkeypatch?*** *Для тимчасової зміни оточення (наприклад, змінних середовища або функцій).*
40. ***Як змінити environment?*** *monkeypatch.setenv("NAME", "value").*
41. ***Як видалити environment?*** *monkeypatch.delenv("NAME", raising=False).*
42. ***Як підмінити function?*** *monkeypatch.setattr(module, "func_name", fake_func).*
43. ***Що таке test double?*** *Загальна назва для об'єктів-заглушок (Mock, Stub, Fake).*
44. ***Що таке Dummy?*** *Об'єкт, який просто передається для заповнення аргументів і не використовується.*
45. ***Що таке Stub?*** *Об'єкт, який повертає заздалегідь підготовлені відповіді.*
46. ***Що таке Fake?*** *Робоча, але спрощена реалізація (наприклад, In-Memory база замість справжньої).*
47. ***Що таке Mock?*** *Об'єкт, який записує всі виклики до нього і дозволяє перевірити взаємодію.*
48. ***Що таке Spy?*** *Mock, який обгортає реальний об'єкт і записує виклики.*
49. ***Для чого Mock?*** *Для імітації зовнішніх систем (API, БД), щоб не робити реальних запитів.*
50. ***Що таке spec?*** *Параметр Mock, який обмежує його лише тими методами, що є в оригінальному класі.*
51. ***Для чого return_value?*** *Вказує, що саме має повернути Mock при виклику.*
52. ***Для чого side_effect?*** *Дозволяє згенерувати виняток або повертати різні значення при повторних викликах.*
53. ***Як перевірити, що викликано 1 раз?*** *mock_obj.method.assert_called_once().*

54. ***Як перевірити parameters?*** *mock_obj.method.assert_called_with(arg1, arg2).*
55. ***Як перевірити, що не викликався?*** *mock_obj.method.assert_not_called().*
56. ***Що таке state testing?*** *Перевірка кінцевого стану об'єкта після виконання.*
57. ***Що таке interaction testing?*** *Перевірка того, як саме об'єкт взаємодіяв з іншими (які методи викликав).*
58. ***Коли interaction виправдане?*** *Коли метод не змінює стан, а лише робить побічну дію (наприклад, відправляє email).*
59. ***Що таке patch?*** *Декоратор або контекстний менеджер для підміни об'єктів.*
60. ***Чому patch там, де використовується?*** *Бо Python шукає імена там, де вони імпортовані, а не там, де створені.*
61. ***Що таке AsyncMock?*** *Mock для асинхронних функцій (які викликаються через await).*
62. ***Як перевірити awaited call?*** *mock_obj.method.assert_awaited_once_with(arg).*
63. ***Що таке flaky test?*** *Тест, який то проходить, то падає без змін у коді (наприклад, через залежність від часу).*
64. ***Чому flaky небезпечні?*** *Вони підривають довіру до тестів, розробники починають ігнорувати падіння.*
65. ***Що таке deterministic test?*** *Тест, який завжди дає однаковий результат за однакових умов.*
66. ***Чому unit не повинен залежати від Internet?*** *Бо це робить його повільним і нестабільним.*
67. ***Коли в integration доцільний real filesystem?*** *Коли тестується саме логіка роботи з файлами (через tmp_path).*
68. ***Чому не потрібно mock-ати все?*** *Тест почне перевіряти Mock-и, а не реальну логіку.*
69. ***Що таке over-mocking?*** *Надмірне використання Mock-ів, коли тест нічого не перевіряє, крім самих заглушок.*
70. ***Що таке coverage?*** *Метрика, що показує відсоток коду, виконаного під час тестів.*
71. ***Що таке statement coverage?*** *Покриття рядків коду.*
72. ***Що таке branch coverage?*** *Покриття гілок умовних операторів (if/else).*
73. ***Чим вони відрізняються?*** *Statement перевіряє, чи виконався рядок, а branch — чи перевірені обидві умови.*
74. ***Як отримати coverage?*** *pytest --cov=package_name.*
75. ***Для чого --cov-report=term-missing?*** *Показує в консолі, які саме рядки не покриті.*
76. ***Для чого HTML coverage?*** *Для візуального перегляду покриття у браузері.*
77. ***Чи гарантує 100% відсутність багів?*** *Ні.*
78. ***Чому assertions важливіші?*** *Бо можна виконати 100% коду, але нічого не перевірити (і тест пройде).*
79. ***Що таке coverage threshold?*** *Мінімальний відсоток, нижче якого тести вважаються невдалими.*

80. ***Чому tests мають бути незалежними?*** *Щоб можна було запускати їх у будь-якому порядку і вони не впливали один на одного.*

18. Висновок

У ході лабораторної роботи було створено повноцінний автоматизований test suite для системи банківських рахунків. Використання pytest дозволило структурувати 21 тест на unit та integration. За допомогою fixtures (conftest.py) та parametrization вдалося уникнути дублювання коду. Ізоляція зовнішніх залежностей була досягнута завдяки Mock та AsyncMock, що дозволило перевірити як успішні сценарії, так і збої через side_effect. Використання tmp_path та monkeypatch забезпечило безпечну роботу з файловою системою та конфігурацією. Аналіз покриття (pytest-cov) підтвердив **95.40% statement coverage** та високий branch coverage, що значно перевищує мінімальну вимогу у 80% і свідчить про високу надійність тестового набору.